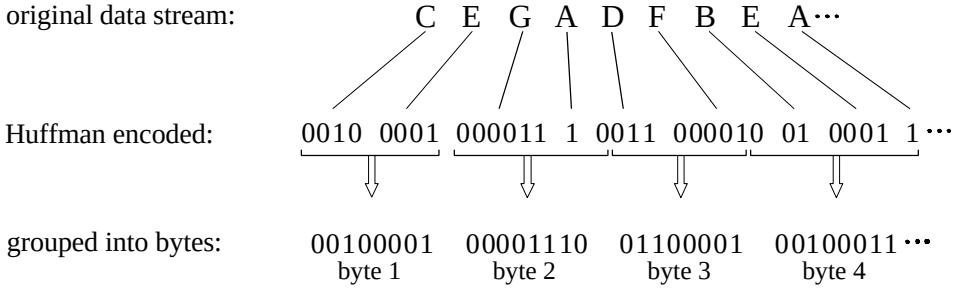
Example Encoding Table

letter	probability	Huffman code
A	.154	1
B	.110	01
C	.072	0010
D	.063	0011
E	.059	0001
F	.015	000010
G	.011	000011

FIGURE 27-3
Huffman encoding. The encoding table assigns each of the seven letters used in this example a variable length binary code, based on its probability of occurrence. The original data stream composed of these 7 characters is translated by this table into the Huffman encoded data. Since each of the Huffman codes is a different length, the binary data need to be regrouped into standard 8 bit bytes for storage and transmission.



looks at the stream of ones and zeros until a valid code is formed, and then starting over looking for the next character. The way that the codes are formed insures that no ambiguity exists in the separation.

A more sophisticated version of the Huffman approach is called **arithmetic encoding**. In this scheme, *sequences* of characters are represented by individual codes, according to their probability of occurrence. This has the advantage of better data compression, say 5-10%. Run-length encoding followed by either Huffman or arithmetic encoding is also a common strategy. As you might expect, these types of algorithms are very complicated, and usually left to data compression specialists.

To implement Huffman or arithmetic encoding, the compression and uncompression algorithms must agree on the binary codes used to represent each character (or groups of characters). This can be handled in one of two ways. The simplest is to use a predefined encoding table that is always the same, regardless of the information being compressed. More complex schemes use encoding optimized for the particular data being used. This requires that the encoding table be included in the compressed file for use by the uncompression program. Both methods are common.

Delta Encoding

In science, engineering, and mathematics, the Greek letter *delta* (Δ) is used to denote the *change* in a variable. The term *delta encoding*, refers to